Chuyên đề: Kỹ thuật Đếm phân phối

Counting Technique trong Competitive Programming

Bài giảng Thuật toán

Mục lục

1. Tư duy đếm phân phối
2. Mảng đếm (Frequency Array)
3. Chuyển đổi góc nhìn
4. Các công thức tổ hợp cần nhớ
5. Dạng bài 1: Đếm số cặp bằng nhau
6. Dạng bài 2: Đếm số bộ ba bằng nhau
7. Dạng bài 3: Đếm số cặp có tổng bằng K
8. Các lỗi thường gặp
9. Mẹo nhận biết bài Counting
10. Tổng kết

Cách tiếp cận ngây thơ (Naive Approach)

Bài toán mở đầu

Cho mảng A gồm N phần tử. Đếm xem trong mảng có bao nhiêu cặp số bằng nhau.

Giải pháp ngây thơ ($O(N^2)$)

Dùng 2 vòng lặp lồng nhau duyệt mọi cặp chỉ số (i, j) với $i < j$:

```
int dem = 0;
for (int i = 0; i < n; i++) {
    for (int j = i + 1; j < n; j++) {
        if (a[i] == a[j]) {
            dem++;
        }
    }
}
```

Hậu quả của cách tiếp cận ngây thơ

Độ phức tạp

Độ phức tạp thời gian: $O(N^2)$

Hậu quả khi thi đấu

- Giới hạn phổ biến: $N = 2 \times 10^5$.
- Số phép tính: $\approx \frac{(2 \times 10^5)^2}{2} = 2 \times 10^{10}$ phép tính.
- Máy tính thông thường chỉ xử lý khoảng 10^8 phép tính / 1 giây.
- **Kết quả:** Chắc chắn bị lỗi **TLE** (Time Limit Exceeded).

Ý tưởng Đếm phân phối (Counting)

Ý tưởng thuật toán

Thay vì hỏi từng cặp phần tử *"Chúng ta có bằng nhau không?"*, hãy **gom tất cả các phần tử có cùng giá trị** vào một "hộp phân phối".

Ghi chú

Khi biết một giá trị X có bao nhiêu phần tử, ta dễ dàng dùng công thức tổ hợp để tính số cặp mà không cần duyệt.

Ví dụ: Mảng `1, 2, 2, 5, 5, 5, 7`

Giá trị (X)	Tiến trình	Tần suất
1		1
2		2
5		3
7		1
Khác		0

Mảng đếm (Frequency Array)

Định nghĩa

Mảng đếm (hay mảng tần suất), thường ký hiệu là `cnt[]` (viết tắt của count), là một mảng bổ trợ dùng để lưu trữ **số lần xuất hiện** của từng giá trị trong cấu trúc dữ liệu ban đầu.

Điều kiện áp dụng

1. Các giá trị trong mảng phải là **số nguyên không âm** (vì chỉ số mảng không thể âm).
2. Giá trị lớn nhất (*MAXV*) phải **đủ nhỏ** để không bị tràn bộ nhớ (**MLE** - Memory Limit Exceeded).
3. Thường giới hạn $MAXV \leq 10^6$ hoặc 10^7 là an toàn cho bộ nhớ ~ 256MB.

Triển khai mã nguồn trong C++

Cách khởi tạo và sử dụng mảng đếm

```cpp
#include <iostream>
#include <vector>
using namespace std;

const int MAXV = 1000000; // Gia tri lon nhat co the xuat hien

int main() {
    int n; cin >> n;
    vector<int> a(n);

    // Khoi tao mang dem kich thuoc (MAXV + 1), gia tri ban dau = 0
    vector<int> cnt(MAXV + 1, 0);

    for (int i = 0; i < n; i++) {
        cin >> a[i];
        cnt[a[i]]++; // Tang tan suat cua gia tri a[i] len 1
    }

    cout << "So lan xuat hien cua so 5 la: " << cnt[5] << endl;
    return 0;
}
```

Chuyển đổi góc nhìn

Kinh nghiệm thi đấu

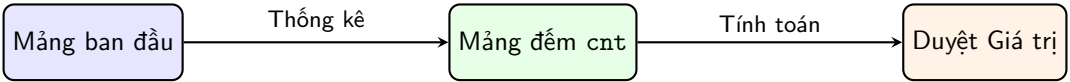
Bí quyết bứt phá trong CP: **Duyệt phần tử** $\rightarrow$ **Duyệt giá trị**

Góc nhìn cũ (Duyệt phần tử)

- Duyệt qua vị trí $0, 1, \ldots, N-1$.
- Vòng lặp phụ thuộc hoàn toàn vào N.
- *Thường dẫn đến* $O(N^2)$.

Góc nhìn mới (Duyệt giá trị)

- Bỏ qua vị trí của các số.
- Duyệt qua các **giá trị** từ $0 \ldots MAXV$.
- *Đưa độ phức tạp về* $O(N+M)$.



Các công thức tổ hợp cần nhớ (1/2)

1. Chọn 2 phần tử giống nhau

Số cặp tạo thành từ n phần tử giống nhau:

$$C_n^2 = \frac{n \times (n-1)}{2}$$

Ví dụ: Có 5 số 7 ($n = 5$).
Số cặp là: $\frac{5 \times 4}{2} = 10$ cặp.

2. Chọn 3 phần tử giống nhau

Số bộ ba tạo thành từ n phần tử giống nhau:

$$C_n^3 = \frac{n \times (n-1) \times (n-2)}{6}$$

Ví dụ: Có 5 số 7 ($n = 5$).
Số bộ ba là: $\frac{5 \times 4 \times 3}{6} = 10$.

Các công thức tổ hợp cần nhớ (2/2)

3. Quy tắc nhân (Khác nhóm)

Chọn 1 phần tử thuộc nhóm A và 1 phần tử thuộc nhóm B ($A \neq B$):

$$\text{Số cặp} = cnt[A] \times cnt[B]$$

Ví dụ: 4 số 3 và 6 số 8. Số cặp $(3, 8)$ là $4 \times 6 = 24$ cặp.

Tổng kết

Thao tác	Số lượng	Công thức	Kiểu dữ liệu
Chọn 2 số giống nhau	n	`n * (n - 1) / 2`	`long long`
Chọn 3 số giống nhau	n	`n * (n - 1) * (n - 2) / 6`	`long long`
Chọn 1 số A, 1 số B	$cnt[A], cnt[B]$	`cnt[A] * cnt[B]`	`long long`

Dạng bài 1: Đếm số cặp bằng nhau

Mô tả bài toán

Cho mảng A gồm N số nguyên dương. Đếm số lượng cặp chỉ số (i, j) sao cho $1 \leq i < j \leq N$ và $A[i] = A[j]$.

Giới hạn: $N \leq 2 \times 10^5$, $A[i] \leq 10^6$.

Ý tưởng thuật toán

1. Duyệt mảng A để xây dựng mảng tần suất `cnt[]`.
2. Duyệt các giá trị v từ $0 \ldots 10^6$.
3. Với mỗi v, nếu `cnt[v]` ≥ 2, cộng dồn C_n^2 vào kết quả.

Mã nguồn C++ - Dạng bài 1

```cpp
#include <iostream>
#include <vector>
using namespace std;
const int MAXV = 1000000;

int main() {
    ios_base::sync_with_stdio(false); cin.tie(NULL);
    int n; cin >> n;
    vector<int> cnt(MAXV + 1, 0);

    for (int i = 0; i < n; i++) {
        int x; cin >> x;
        cnt[x]++;
    }

    long long answer = 0;
    // Duyet theo GIA TRI tu 0 den MAXV
    for (int v = 0; v <= MAXV; v++) {
        if (cnt[v] >= 2) {
            long long freq = cnt[v];
            answer += freq * (freq - 1) / 2; // Cong thuc C(n, 2)
        }
    }
    cout << answer << "\n";
    return 0;
}
```

Đánh giá độ phức tạp - Dạng bài 1

Độ phức tạp

- **Thời gian (Time Complexity):** $O(N+M)$ với $M = MAXV = 10^6$.
- Đọc dữ liệu mất $O(N)$, duyệt giá trị tính toán mất $O(M)$.
- Tổng thời gian ≈ 0.05 giây (Rất tối ưu).

Ghi chú

Không gian (Space Complexity): $O(M)$ để lưu trữ mảng đếm `cnt`, hoàn toàn nằm trong giới hạn bộ nhớ cho phép (chỉ tốn vài MB).

Dạng bài 2: Đếm số bộ ba bằng nhau

Mô tả bài toán

Cho mảng A gồm N số nguyên dương. Đếm số lượng bộ ba chỉ số (i, j, k) sao cho $1 \leq i < j < k \leq N$ và $A[i] = A[j] = A[k]$.
Giới hạn: $N \leq 2 \times 10^5$, $A[i] \leq 10^6$.

Kinh nghiệm thi đấu

Phân tích:
Tương tự dạng 1, điểm khác biệt duy nhất là bốc ra 3 phần tử giống nhau. Ta áp dụng công thức C_n^3 thay vì C_n^2.

Mã nguồn C++ - Dạng bài 2

```cpp
#include <iostream>
#include <vector>
using namespace std;
const int MAXV = 1000000;

int main() {
    ios_base::sync_with_stdio(false); cin.tie(NULL);
    int n; cin >> n;
    vector<int> cnt(MAXV + 1, 0);

    for (int i = 0; i < n; i++) {
        int x; cin >> x;
        cnt[x]++;
    }

    long long answer = 0;
    for (int v = 0; v <= MAXV; v++) {
        if (cnt[v] >= 3) {
            long long freq = cnt[v];
            answer += freq * (freq - 1) * (freq - 2) / 6; // C(n, 3)
        }
    }
    cout << answer << "\n";
    return 0;
}
```

Dạng bài 3: Đếm số cặp có tổng bằng K

Mô tả bài toán

Cho mảng A ($N \leq 2 \cdot 10^5$, $A[i] \leq 10^6$) và số $K \leq 2 \cdot 10^6$. Đếm số cặp (i, j) sao cho $i < j$ và $A[i] + A[j] = K$.

Yêu cầu: Không dùng Map, Con trỏ, hay Chặt nhị phân.

Ý tưởng thuật toán

Duyệt theo góc nhìn giá trị: Nếu chọn một số có giá trị là v, số còn lại bắt buộc phải là $K - v$. Chúng ta chỉ cần duyệt v từ 0 đến $K/2$ (để tránh đếm trùng).

Phân tích Ý tưởng - Dạng bài 3

Duyệt qua mọi giá trị v khả dĩ từ 0 đến $K/2$:

Trường hợp 1: $v \neq K - v$

Hai giá trị khác hẳn nhau. Theo quy tắc nhân, số cặp tạo thành là:

$$cnt[v] \times cnt[K - v]$$

Trường hợp 2: $v = K - v$ (Khi K chẵn và $v = K/2$)

Hai phần tử lấy từ cùng một nhóm tần suất. Chọn 2 phần tử từ nhóm này, áp dụng công thức C_n^2:

$$\frac{cnt[v] \times (cnt[v] - 1)}{2}$$

Mã nguồn C++ - Dạng bài 3

```cpp
#include <iostream>
#include <vector>
using namespace std;
const int MAXV = 2000000;

int main() {
    ios_base::sync_with_stdio(false); cin.tie(NULL);
    int n, k; cin >> n >> k;
    vector<int> cnt(MAXV + 1, 0);

    for (int i = 0; i < n; i++) {
        int x; cin >> x;
        if (x <= MAXV) cnt[x]++;
    }

    long long answer = 0;
    for (int v = 0; v <= k / 2; v++) {
        int target = k - v;
        if (v == target) {
            long long freq = cnt[v];
            answer += freq * (freq - 1) / 2; // Cung nhom
        } else {
            long long freq1 = cnt[v], freq2 = cnt[target];
            answer += freq1 * freq2; // Khac nhom
        }
    }
```

Các lỗi thường gặp (1/3)

1. Tràn số (Integer Overflow) khi nhân tần suất

- **Sai:** `answer += cnt[v] * (cnt[v] - 1)/ 2;`
- **Đúng:** `answer += (long long)cnt[v] * (cnt[v] - 1)/ 2;`

Ghi chú

Giải thích: Nếu `cnt[v]` kiểu `int`, phép nhân giữa 2 biến `int` tính toán theo kiểu `int` trước khi gán. Nếu $cnt[v] = 10^5$, kết quả 10^{10} vượt quá giới hạn 2×10^9, sinh ra lỗi sai số âm.

Các lỗi thường gặp (2/3)

2. Truy cập bộ nhớ ngoài mảng (Out of Bounds)

- **Sai:** Khai báo `int cnt[1000];` nhưng đầu vào có giá trị 2000. Gọi `cnt[2000]++` sẽ gây **Runtime Error (SIGSEGV)**.
- **Giải pháp:** Luôn kiểm tra kỹ giới hạn đề bài để khai báo kích thước mảng chuẩn xác nhất.

3. Quên khởi tạo giá trị 0 ban đầu

- **Sai:** `int cnt[MAXV];` (biến cục bộ). Mảng sẽ chứa rác.
- **Đúng:** Dùng `vector<int> cnt(MAXV + 1, 0);` hoặc khai báo mảng tĩnh toàn cục.

Các lỗi thường gặp (3/3)

4. Đếm trùng lặp (Double Counting)

Đặc biệt trong bài toán tìm cặp có tổng bằng K.

- **Sai:** Duyệt giá trị v từ $0 \dots K$.
  $\rightarrow$ Cặp $(v, K - v)$ bị tính 2 lần (lần 1 ở v, lần 2 ở $K - v$).
- **Đúng:** Chỉ duyệt v tới giá trị $K/2$ và tách biệt trường hợp chẵn lẻ như Dạng 3.

Mẹo nhận biết bài toán Counting (1/2)

Khi đọc đề, hãy tìm kiếm các manh mối (clues) sau:

Mẹo (Tips)

Đề bài yêu cầu tính **số lượng**, đếm số cặp, số bộ ba, hoặc tìm tần suất (phần tử xuất hiện nhiều nhất, phần tử duy nhất,...).

Mẹo (Tips)

Số lượng N rất lớn ($10^5 \dots 10^6$), nhưng **giá trị** phần tử $A[i]$ bị giới hạn nhỏ ($A[i] \leq 10^5 \dots 10^6$). Đây là đất diễn lý tưởng của mảng đếm.

Mẹo nhận biết bài toán Counting (2/2)

Mẹo (Tips)

Kết quả bài toán không thay đổi nếu ta **hoán đổi vị trí** các phần tử. (Bản chất nằm ở giá trị, không phải vị trí).

Mẹo (Tips)

Duyệt trâu sinh ra $O(N^2)$ quá lớn và bạn bắt buộc phải tìm cách quy đổi về thống kê tần suất để tính bằng phép toán $O(1)$.

Tổng kết chuyên đề

Tổng kết

1. **Mảng đếm `cnt[x]`**: Công cụ lưu trữ tần suất $O(1)$. Đòi hỏi miền giá trị phần tử nhỏ ($M \leq 10^6$).
2. **Sự chuyển đổi góc nhìn**: Bí kíp tối ưu là chuyển từ Duyệt phần tử (theo N) sang **Duyệt giá trị** (theo M) $\rightarrow$ Tăng tốc từ $O(N^2)$ về $O(N + M)$.
3. **Toán tổ hợp**: Nền tảng sức mạnh đếm. Nắm vững C_n^2, C_n^3 và quy tắc nhân để giải quyết các bài đếm cặp, đếm bộ, hình học cực kỳ thanh thoát.